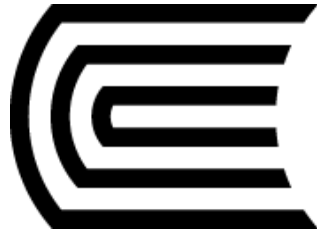


"Año de la Esperanza y el Fortalecimiento de la Democracia"



**Universidad
Continental**

Estadística y Probabilidades:

NRC: 28651

Docente :

Rosario Delia Osorio Contreras

Grupo:

"Grupo 4"

Trabajo Grupal:

INTEGRANTES:

- CHUQUIN CANCHUMANYA CARLOS ANDRES
- TERREL SÁNCHEZ FABRISIO RAMÓN
- DIAZ ALTAMIRANO BRITTANY

HUANCAYO- PERÚ 2026

Enlace al repositorio de Github:

<https://github.com/61207267-ui/Estructura-de-datos.git>

ÍNDICE

CAPÍTULO 1: Análisis del Problema	1
1.1. Descripción del problema	1
1.2. Requerimientos del sistema	2
1.2.1. Funcionales	2
1.2.2. No funcionales	2
1.3. Estructuras de datos propuestas	2
1.4. Justificación de la elección	2
Capítulo 2: Diseño de la Solución	3
2.1. Descripción de estructuras de datos y operaciones:	3
2.2. Algoritmos principales:	3
2.2.1. Pseudocódigo para agregar proceso.	3
2.2.2. Pseudocódigo para cambiar el estado del proceso.	3
2.3. Diagramas de Flujo	3
2.4.. Justificación del diseño:	3
Capítulo 3: Solución Final	4
3.1. Código limpio, bien comentado y estructurado.	4
3.2. Capturas de pantalla de las ventanas de ejecución con las diversas pruebas de validación	4
de datos	4
3.3. Manual de usuario	4
Capítulo 4: Evidencias de Trabajo en Equipo	4
4.1. Repositorio con Control de Versiones	4
4.1.1. Historial de ramas y fusiones si es aplicable.	4
4.2.2. Evidencia por cada integrante del equipo.	4
4.2.3. Enlace a la herramienta colaborativa	4
4.2. Plan de Trabajo y Roles Asignados	4
4.2.1. Documento inicial donde se asignan tareas y responsabilidades.	4
4.2.2. Cronograma con fechas límite para cada entrega parcial.	4
4.2.3. Registro de reuniones o comunicación del equipo (Actas de reuniones.).	4

CAPÍTULO 1: Análisis del Problema

1.1. Descripción del problema

El presente proyecto aborda la necesidad de simular el comportamiento de un **Sistema de Gestión de Procesos** en un Sistema Operativo (SO) mediante el uso de la programación en C++. Como sabemos, la computación moderna ha podido resolver estos procesos de forma interna y eficiente; sin embargo, el propósito de este proyecto es simular estos tres componentes principales: el **Gestor de Procesos**, el **Planificador de CPU** y el **Gestor de Memoria** para un mejor entendimiento de su funcionamiento lógico haciendo uso exclusivo de estructuras dinámicas lineales desarrolladas desde cero.

A través de esta simulación, entenderemos la complejidad que tienen estos procesos durante todo su ciclo de vida, el cual incluye las etapas de **creación, espera, ejecución y finalización**. Sin un software de control como este, el sistema operativo real sufriría graves errores operativos. Por ejemplo, a base de la ausencia de una cola de prioridades, las tareas críticas se quedarían congeladas sin saber cual es la más urgente o de prioridad ; asimismo, la falta de control en la asignación y liberación de memoria provocaría un colapso por desbordamiento de recursos . Por lo tanto, este sistema se vuelve indispensable para comprender la administración y optimización de los **recursos del software**

1.2. Requerimientos del sistema

1.2.1. Funcionales

- **Registrar e Insertar Procesos:** El sistema debe permitir dar de alta nuevos procesos asignándoles un identificador único, nombre y nivel de prioridad, almacenándolos de forma dinámica.

- **Búsqueda y Modificación:** El usuario podrá buscar procesos específicos por su ID o nombre, además de contar con la opción de modificar su nivel de prioridad en tiempo real.
- **Eliminar Procesos:** El sistema debe ser capaz de remover un proceso específico del registro general cuando sea solicitado por el usuario.
- **Planificación de CPU:** El sistema debe encolar los procesos según su nivel de prioridad y simular su posterior procesamiento mediante el desencholamiento ordenado.
- **Gestión de Memoria:** El sistema debe asignar bloques de memoria a los procesos activos mediante una pila y liberarlos secuencialmente conforme el proceso termine su ejecución.
- **Persistencia de Datos:** El sistema debe incluir la funcionalidad de guardar el estado actual de los procesos en un archivo local y cargarlos automáticamente al iniciar la aplicación.
- **Validación de Entradas de Datos:** El sistema debe evaluar y controlar activamente todas las entradas del usuario por consola. Debe restringir el ingreso de datos (por ejemplo, impedir caracteres alfabéticos en el ID o la prioridad, y evitar prioridades fuera del rango permitido).

1.2.2. No funcionales

- **Lenguaje y Entorno de Desarrollo:** La aplicación debe ser desarrollada en lenguaje C++ y ser completamente ejecutable utilizando el entorno DEV C++.
- **Estructuras Dinámicas Propias:** Queda estrictamente prohibido el uso de librerías predefinidas para las estructuras de datos. Toda la lógica de punteros y nodos debe implementarse desde cero. Dentro de este apartado se desarrollarán:

☐ **Listas Enlazadas:** Se utilizarán para simular el **registro global de procesos**, permitiendo insertar, buscar, modificar o eliminar nodos en memoria dinámica de forma flexible según los requerimientos del usuario.

☐ **Colas:** Se utilizarán para simular el **Planificador de CPU**, donde los procesos se encolarán de manera ordenada de acuerdo a su nivel de prioridad para luego ser ejecutados secuencialmente mediante un orden de salida.

☐ **Pilas:** Se utilizarán para simular el **Gestor de Memoria**, donde los bloques de memoria requeridos por cada proceso se apilarán y desapilarán dinámicamente, representando el comportamiento físico de asignación y liberación de recursos.

- **Uso de Structs:** La información de los procesos, la memoria y los nodos de control se organizará utilizando estructuras (**struct**) para garantizar un código limpio y estructurado de forma modular.
- **Interfaz de Usuario:** El control e interacción con el sistema se realizará mediante una interfaz de consola interactiva basada en menús.
- **Control de Versiones (GitHub):** El código fuente debe estar alojado en un repositorio Git que cuente con un historial de commits para evidenciar el trabajo equitativo y colaborativo de todos los miembros del equipo.
- **Uso de Constructores en los Nodos:** Toda estructura de control y nodo dinámico implementado debe contar obligatoriamente con métodos constructores explícitos. Estos se encargarán de automatizar la asignación de datos y de inicializar por defecto los punteros de enlace a **NULL** (o punteros nulos) para el uso de listas.

1.3. Estructuras de datos propuestas

Para cumplir con el objetivo de este proyecto, el sistema a realizar se dividirá en tres procesos independientes:

1. **Lista Enlazada Simple Dinámica (Gestor de Procesos):** Se encargará del registro global y control de las tareas creadas. Administrará el ciclo de vida general de cada proceso permitiendo las operaciones de inserción, eliminación, búsquedas por identificador o nombre, y la actualización de sus propiedades en memoria dinámica **(todo esto a base de nodos y punteros)**.
2. **Cola de Prioridad Dinámica (Planificador de CPU):** Se encargará de simular el orden de procesamiento y ejecución en el procesador. Su función principal será organizar los elementos de acuerdo a su nivel de importancia (prioridad alta o baja) y quitarlos de la lista de prioridad secuencialmente para simular el uso del tiempo de la CPU.
3. **Pila Dinámica (Gestor de Memoria):** Se encargará de simular la administración de la memoria RAM asignada a los procesos activos. Su rol consistirá en otorgar bloques de memoria a los procesos mediante operaciones de apilamiento **(push)** y liberarlos del sistema una vez concluidos mediante el desapilamiento **(pop)**.

1.4. Justificación de la elección

La razón por la cual se justifica el uso de estas estructuras dinámicas es que se adaptan al tipo de lógica que demanda este proyecto, además de poder emular el comportamiento real de los subprocesos internos de un sistema operativo, gracias al uso de memoria dinámica basada en nodos y punteros.

La elección de cada estructura se fundamenta detalladamente a continuación:

1. **Justificación de la Lista Enlazada:** Se seleccionó para el Gestor de Procesos debido a su flexibilidad para manipular datos en cualquier posición de la memoria. En un sistema operativo, los procesos no se crean ni se eliminan en un orden fijo; los usuarios abren y cierran tareas aleatoriamente. La lista enlazada permite insertar nuevos procesos, buscarlos por ID, modificar sus atributos o eliminarlos en tiempo real con un costo computacional óptimo, sin necesidad de reorganizar o desplazar bloques fijos de memoria.
2. **Justificación de la Cola de Prioridad:** Su elección es indispensable para el Planificador de CPU ya que altera la **lógica convencional FIFO (First In, First Out)** para adaptarse a las necesidades de un procesador moderno. No todas las tareas del sistema tienen la misma urgencia; los procesos del núcleo del sistema operativo requieren atención inmediata antes que las aplicaciones del usuario. Esta estructura organiza dinámicamente los elementos colocando al frente aquellos con mayor nivel de prioridad, garantizando que la simulación de ejecución de la CPU atienda primero lo más crítico.
3. **Justificación de la Pila:** El uso de esta estructura se justifica plenamente para el Gestor de Memoria porque sigue de forma estricta **el principio LIFO (Last In, First Out)**, el cual es el estándar técnico utilizado por la memoria (*stack*) física de las computadoras. Cuando un proceso requiere subrutinas o solicita recursos, estos bloques de memoria se apilan cronológicamente. Al finalizar la tarea, el sistema operativo debe liberar primero el último bloque asignado para evitar fugas de memoria, haciendo que las operaciones de **apilar (push) y desapilar (pop)** reflejen la gestión de recursos del software.

Capítulo 2: Diseño de la Solución

INTEGRANTE	ROL
FABRISIO TERREL	PSEUDOCÓDIGO
CHUQUIN CANCHUMANYA	DIAGRAMA DE FLUJO

BRITANY DÍAS	JUSTIFICACIÓN DEL DISEÑO
--------------	--------------------------

2.1. Descripción de estructuras de datos y operaciones:

2.1.1. Definición de las Estructuras de Datos

Lista Enlazada (Gestor Global de Procesos): Es una estructura de datos lineal y dinámica compuesta por una secuencia de nodos conectados entre sí mediante punteros. En este proyecto, se define mediante un puntero inicial llamado **siguiente** que apunta al primer elemento. Se utiliza como el **Registro Central del Sistema Operativo**, permitiendo almacenar, buscar, recorrer y eliminar de forma secuencial todos los procesos creados en el sistema, sin importar el orden en que llegaron.

Cola de Prioridad (Planificador de CPU): Es una estructura de datos lineal que sigue el principio FIFO (*First In, First Out* - Primero en entrar, primero en salir), pero modificada para ordenar sus elementos según un atributo de importancia. Se define y controla mediante dos punteros simultáneos: **frente** (salida del elemento con mayor prioridad) y **final** (punto de inserción). En el simulador, representa la **Fila de Espera del Procesador (CPU)**, donde los procesos aguardan su turno de ejecución ordenados estrictamente por su nivel de prioridad.

Pila Dinámica (Gestor de Memoria RAM): Es una estructura de datos lineal que se rige bajo el principio LIFO (*Last In, First Out* - Último en entrar, primero en salir). A diferencia de las anteriores, su acceso está restringido y se controla de forma exclusiva mediante un único puntero denominado **tope** (cima de la estructura). En esta simulación, actúa como el **Gestor de la Memoria RAM**, donde la asignación y liberación de bloques de memoria se realiza apilando y desapilando los registros en el tope, simulando el comportamiento real del *Stack* de memoria de un sistema operativo.

2.1.2. Operaciones Fundamentales del Sistema

A. Operaciones del Gestor de Procesos (Lista Enlazada)

- **insertarProcesoFinal:** Añade un nuevo proceso al final de la lista enlazada, simulando su registro general en el sistema operativo. Se encarga de enganchar de forma dinámica cada nueva aplicación que entra en ejecución al historial global del sistema.
- **buscarProceso:** Recorre los nodos de la lista mediante un puntero auxiliar para localizar un proceso por su identificador único. Se encarga de realizar la auditoría o inspección de los datos de un proceso específico dentro del registro de control del sistema operativo.
- **modificarPrioridad:** Ubica el nodo correspondiente y actualiza su atributo de prioridad en tiempo real. Se encarga de alterar la urgencia o el nivel de importancia de una tarea de acuerdo con las demandas dinámicas del entorno del software.
- **eliminarProceso:** Desenlaza el nodo de la lista, reestructura los punteros adyacentes y libera la memoria RAM utilizada mediante el operador **delete**. Se encarga de cerrar de forma definitiva una aplicación, asegurando que no queden registros sueltos o basura en la lista global del sistema operativo.

B. Operaciones del Planificador de CPU (Cola de Prioridad)

- **encolarProcesoConPrioridad:** Inserta un proceso en la cola, pero a diferencia de una cola común, este se posiciona recorriendo la estructura hasta encontrar su lugar correspondiente según su nivel de prioridad (los de mayor prioridad van al frente). Se encarga de ordenar de forma jerárquica la fila de espera para optimizar los tiempos de respuesta del procesador.

- **desencolarProceso:** Extrae el proceso que se encuentra al inicio de la cola (el de mayor prioridad listo para ser ejecutado), simulando el uso de los ciclos del procesador. Se encarga de despachar la tarea más urgente y transferirle el control inmediato del hardware de la CPU.
- **mostrarCola:** Realiza un recorrido secuencial desde el frente hasta el final para visualizar en pantalla los procesos en espera de ejecución. Se encarga de simular el monitor de rendimiento visual del sistema operativo, permitiendo observar las tareas pendientes en tiempo real.

C. Operaciones del Gestor de Memoria (Pila)

- **pushMemoria:** Apila un bloque de memoria asignado al proceso en la cima de la estructura (Stack), reservando lógicamente el recurso del software. Se encarga de registrar la ocupación física del espacio de memoria RAM asignado al último proceso que acaba de ser cargado en el sistema operativo.
- **popMemoria:** Libera el último bloque de memoria asignado que se encuentra en la cima de la pila (*Last In, First Out*), simulando la finalización exitosa de la tarea. Se encarga de limpiar el espacio en el tope del Stack, devolviendo el control de ese fragmento de RAM que fue utilizado.
- **pilaVacía:** Función de validación de tipo booleana que verifica si el puntero Tope apunta a NULO, indicando que la memoria asignada se encuentra completamente liberada. Se encarga de verificar que el gestor de memoria RAM se encuentre en un estado limpio, seguro y sin fugas de memoria remanentes.

2.2. Algoritmos principales:

2.2.1. *Pseudocódigo para agregar proceso.*

Algoritmo agregarProceso(*id, nombre, prioridad*)

 // Crear nuevo nodo

 Crear NodoLista nuevo

 nuevo.id = id

 nuevo.nombre = nombre

 nuevo.prioridad = prioridad

 nuevo.siguiente = NULL

 Si (listalInicio == NULL) Entonces

 listalInicio = nuevo

 Sino

 temporal = listalInicio

 Mientras (temporal.siguiente != NULL) Hacer

 temporal = temporal.siguiente

 FinMientras

 temporal.siguiente = nuevo

 FinSi

 Mostrar "Proceso agregado correctamente"

FinAlgoritmo

2.2.2. Pseudocódigo para cambiar el estado del proceso.

Algoritmo modificarPrioridad(id, nuevaPrioridad)

Si (listalInicio == NULL) Entonces

Mostrar "Lista vacía. No hay procesos para modificar."

Retornar

FinSi

temporal = listalInicio

Encontrado = Falso

Mientras (temporal != NULL) Hacer

Si (temporal.id == id) Entonces

temporal.prioridad = nuevaPrioridad

Encontrado = Verdadero

Romper

FinSi

temporal = temporal.siguiente

FinMientras

Si (Encontrado == Verdadero) Entonces

Mostrar "Prioridad del proceso", id, "actualizada correctamente a",
nuevaPrioridad

Sino

Mostrar "Proceso con ID", id, "no encontrado"

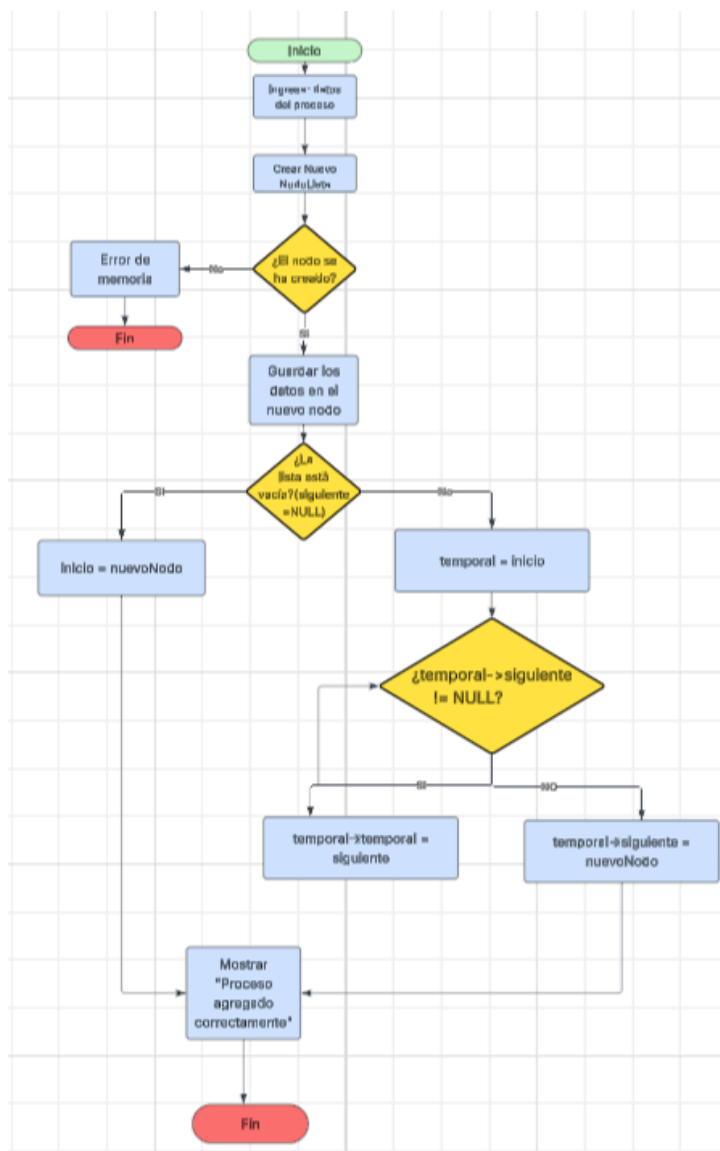
FinSi

FinAlgoritmo

2.3. Diagramas de Flujo

A continuación se presentan los diagramas de flujo correspondientes a las operaciones principales del sistema:

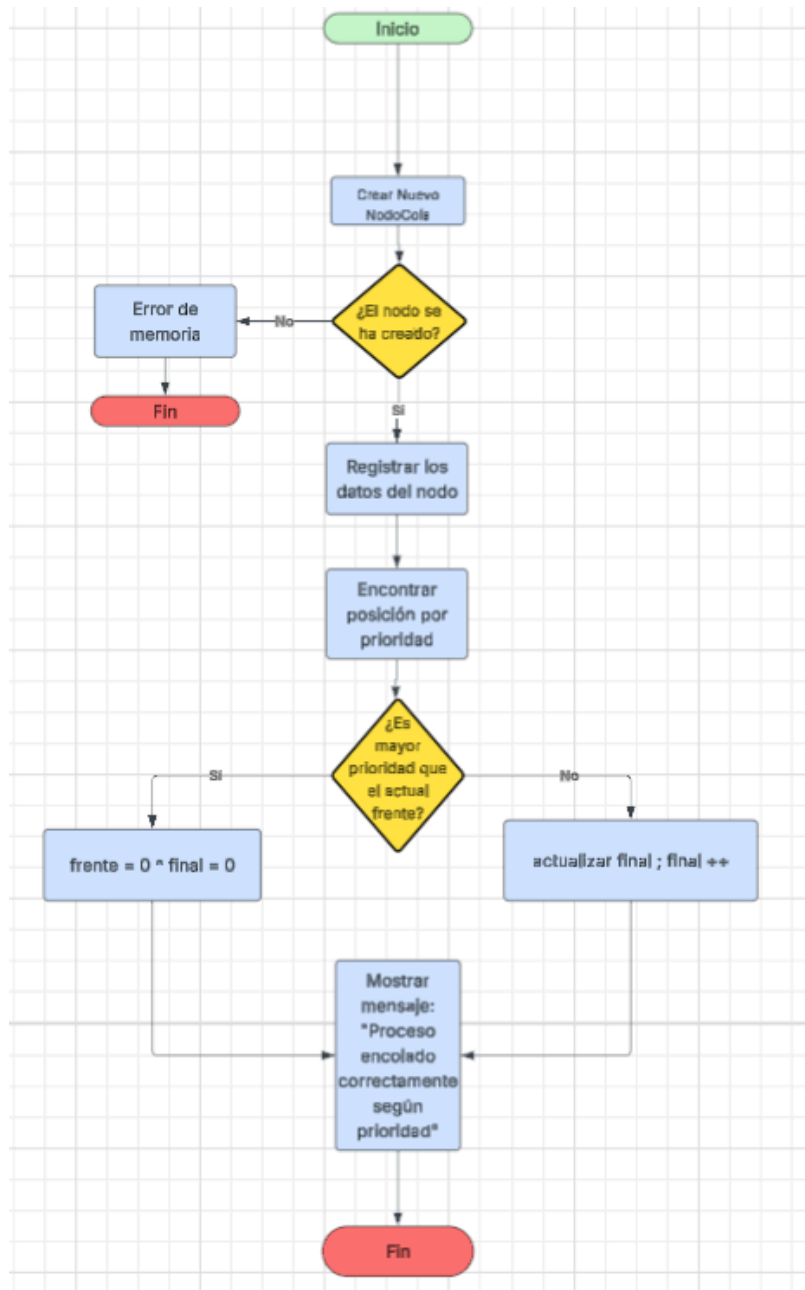
Diagrama 1: Agregar Proceso en Lista Enlazada (Gestor de Procesos)



Este diagrama muestra detalladamente el proceso de inserción de un nuevo proceso en la lista enlazada simple. Se consideran dos casos principales: cuando la lista se encuentra vacía y cuando ya contiene elementos. El algoritmo recorre

la lista hasta encontrar el final para realizar la inserción manteniendo el orden de llegada.

Diagrama 2: Encolar Proceso según Prioridad (Planificador de CPU)



Este diagrama ilustra el flujo para encolar un proceso respetando su nivel de prioridad. La operación recorre la cola para encontrar la posición correcta según el nivel de prioridad del proceso, garantizando que los procesos de mayor prioridad sean atendidos primero en el planificador de CPU.

2.4. Justificación del diseño:

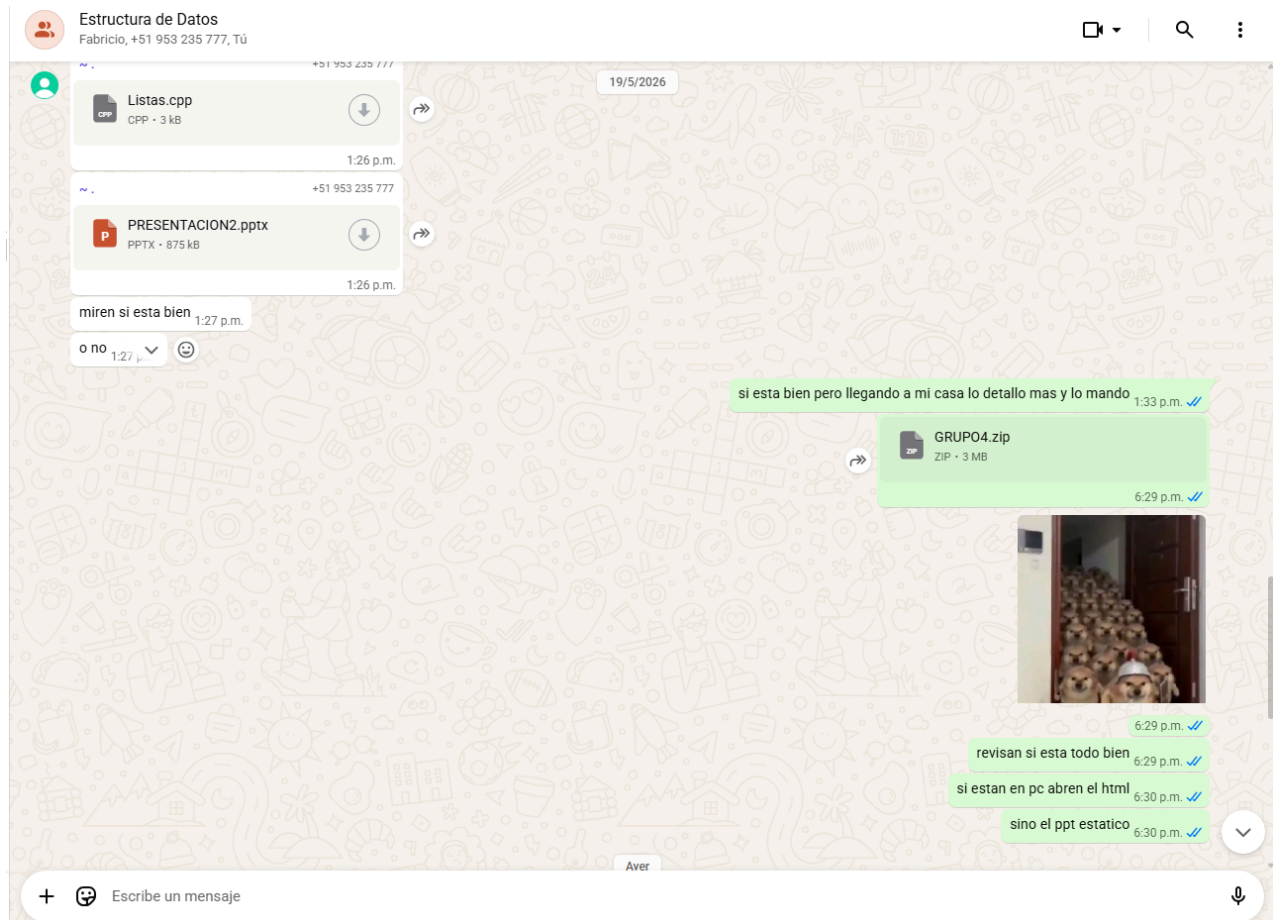
El diseño de este simulador fue desarrollado utilizando estructuras de datos dinámicas con el objetivo principal de superar las limitaciones técnicas de los arreglos estáticos, los cuales exigen trabajar con un tamaño definido y rígido de memoria. Este enfoque dinámico permite gestionar los recursos del software de una manera mucho más flexible y adaptable durante el tiempo de ejecución.

La lógica y el propósito detrás de la selección de cada estructura se justifican de la siguiente manera:

- **Lista Enlazada Simple (Módulo de Registro):** En este componente se estableció el uso de un único puntero llamado **inicio**, con el cual se logra un crecimiento ilimitado y ordenado en la estructura, simulando perfectamente el escenario real donde el usuario crea nuevos procesos de forma constante. Al ser una estructura puramente dinámica, facilita enormemente la ejecución de operaciones esenciales de administración, tales como la búsqueda, modificación, visualización y eliminación de tareas sin restricciones de espacio.
- **Cola Dinámica (Planificador de CPU):** Para este módulo se diseñó una cola especial donde se rompe el famoso principio FIFO (*First In, First Out*). Esta alteración es fundamental, ya que el sistema tiene que simular que el proceso con mayor prioridad debe ser siempre el más próximo a ser resuelto por el procesador. Bajo esta lógica, se garantiza que **los procesos críticos del sistema operativo sean solucionados con total prioridad**, antes que otros procesos menores o secundarios del usuario.
- **Pila Dinámica (Gestor de Memoria):** En el control del almacenamiento se utilizó estrictamente la lógica de una estructura LIFO (*Last In, First Out*). Con

la ayuda del puntero **tope**, este concepto se adapta con gran precisión al comportamiento de la memoria real de un sistema operativo, resultando ser la opción más eficaz para representar cómo se asignan y liberan de forma secuencial los bloques de memoria asignados a ciertas tareas del programa.

Evidencias de trabajo colaborativo:



Capítulo 3: Solución Final

3.1. Código limpio, bien comentado y estructurado.

3.2. Capturas de pantalla de las ventanas de ejecución con las diversas pruebas de validación

de datos

3.3. Manual de usuario

Capítulo 4: Evidencias de Trabajo en Equipo

4.1. Repositorio con Control de Versiones

4.1.1. Historial de ramas y fusiones si es aplicable.

4.2.2. Evidencia por cada integrante del equipo.

4.2.3. Enlace a la herramienta colaborativa

4.2. Plan de Trabajo y Roles Asignados

4.2.1. Documento inicial donde se asignan tareas y responsabilidades.

4.2.2. Cronograma con fechas límite para cada entrega parcial.

4.2.3. Registro de reuniones o comunicación del equipo (Actas de reuniones.).